STAT 6820: Assignment 5

Hyrum Hansen

2023-03-24

Problem 1:

Using the D-criteria implementation found in the code appendix, a D score was computed for the following matrix:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & -1 & 1 & -1 & 1 \\ 1 & -1 & -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & 1 & 1 & -1 \\ -1 & -1 & -1 & 1 & -1 & 1 \\ 1 & 1 & 1 & -1 & -1 & 1 \\ 1 & -1 & 1 & 1 & 1 & 1 \\ -1 & -1 & -1 & -1 & 1 & 1 \\ -1 & 1 & -1 & -1 & 1 & 1 \\ 1 & 1 & -1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 & -1 & -1 \\ -1 & -1 & 1 & -1 & -1 & -1 \\ 1 & -1 & -1 & -1 & -1 & -1 \end{bmatrix}$$

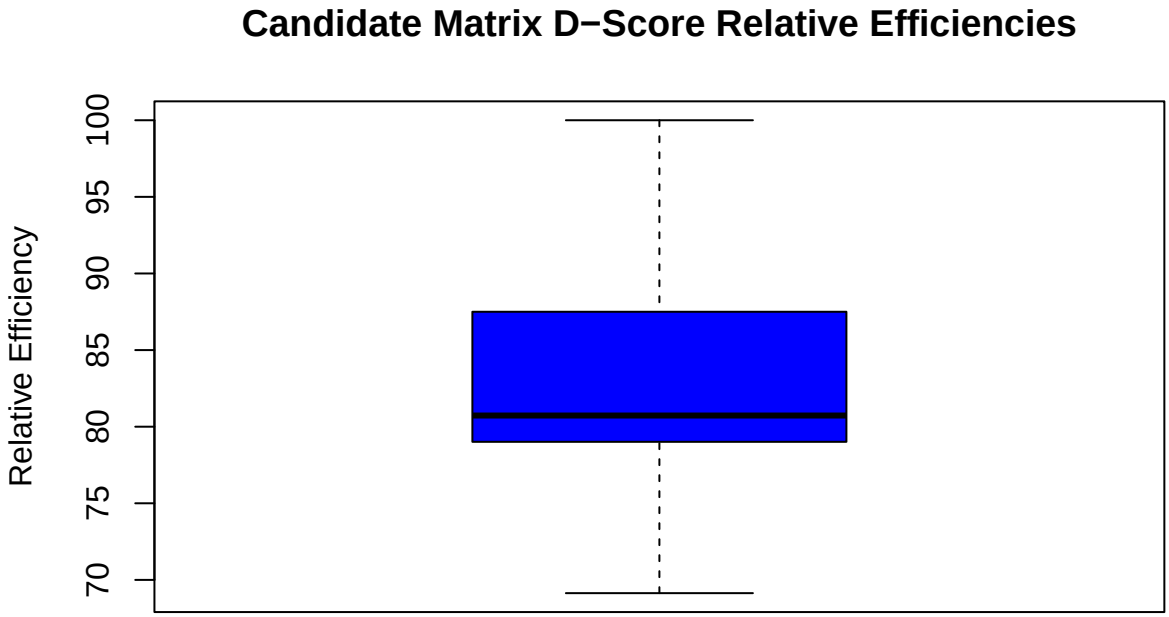
.

The calculation was

$$D(\mathbf{X}) = |X^T X| \approx 339738624$$

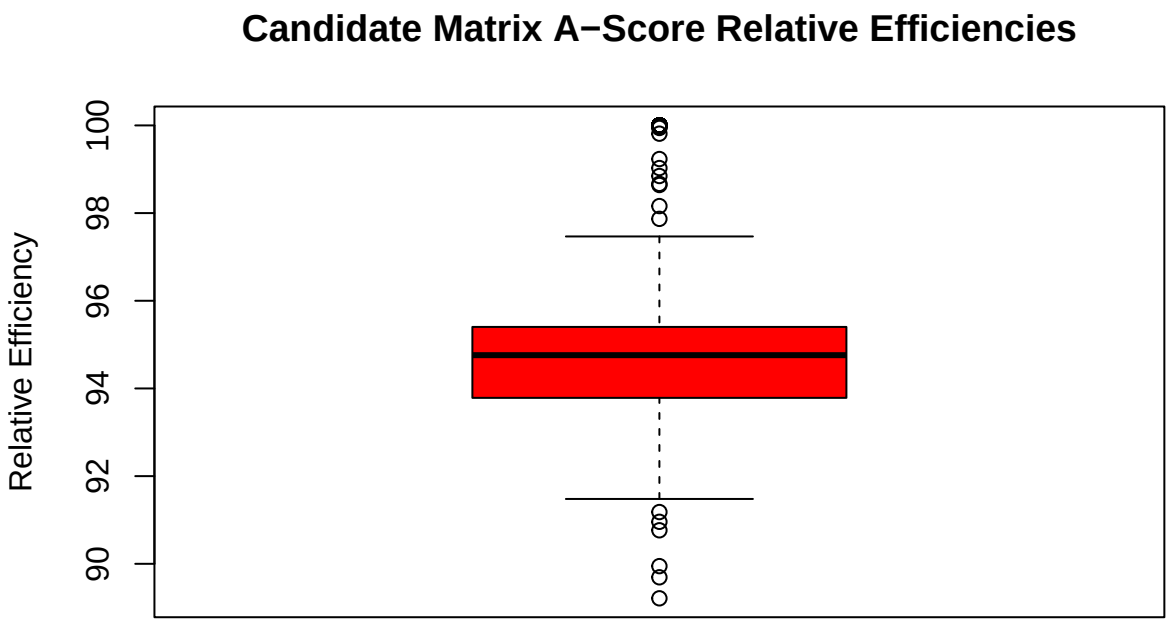
Problem 2:

2000 iterations of coordinate exchange resulted in an optimal design with a D score of 339738624. This is exactly as good as the design proposed by Peter Goos and Brad Jones in the Optimal Design of Experiments textbook.



Problem 3:

The A optimal design gave an A score of score of 0.7083. Once again, this was pretty much identical to the author's proposed design. (0.7083) The best of the candidate designs had A scores roughly equivalent to the A score of the author's D optimal design.



Problem 4:

a. $D(\mathbf{X}_A^*) = 339738624$
   Relative Efficiency: 100%

b. $A(\mathbf{X}_D^*) = 0.7083333$
   Relative Efficiency: 100%

Interestingly, these matrices performed equally well on the other's criteria.

Problem 5:

Coordinate exchange was run to determine the optimal matrix with candidate values $\{-1, -0.9, -0.8, ..., 0.8, 0.9, 1\}$. The optimal designs produced with this new method scored equivalently well for both A and D scores, and their values were commensurate with the optimal designs generated using candidate values $\{-1, 1\}$. I feel somewhat surprised that more swaps didn't generate higher quality designs. I suppose this means the functions being optimized over have optimum values where factor levels are set to extremes.

Problem 6:

The final D optimal matrix was

$$X_D^* = \begin{bmatrix} 1 & -1 & -1 & -1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 & -1 & 1 \\ -1 & -1 & 1 & -1 & 1 & 1 \\ 1 & -1 & -1 & 1 & -1 & 1 \\ 1 & -1 & 1 & 1 & -1 & -1 \\ -1 & 1 & 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 & -1 \\ -1 & 1 & -1 & 1 & -1 & 1 \\ -1 & -1 & -1 & 1 & 1 & -1 \\ 1 & 1 & -1 & -1 & 1 & -1 \end{bmatrix}$$

and the final A optimal matrix was

$$X_A^* = \begin{bmatrix} 1 & -1 & -1 & -1 & -1 & 1 \\ 1 & 1 & -1 & 1 & 1 & -1 \\ -1 & -1 & -1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 & -1 & -1 \\ -1 & -1 & 1 & 1 & -1 & -1 \\ -1 & 1 & 1 & 1 & -1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 & 1 \\ 1 & 1 & 1 & -1 & -1 & -1 \\ -1 & -1 & -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & -1 & -1 & 1 \end{bmatrix}$$

Because these matrices perform equally well on both criteria it's difficult to reccomend one over the other.

```r
knitr::opts_chunk$set(echo = FALSE)
library(matlib)
library(xtable)

##########################
## FUNCTION DEFINITIONS ##
##########################

# This function computes the D score for a given matrix
compute_D <- function(X){

  # First expand the design into a model matrix
  intercept <- rep(1, nrow(X))
  interaction <- X[,2]*X[,3]
  model_matrix <- cbind(intercept, X, interaction)
  # Then compute the D score for the candidate design
  return(det(t(model_matrix)%*%model_matrix))
}

# This function computes the A score for a given matrix
compute_A <- function(X){

  # First expand the design into a model matrix
  intercept <- rep(1, nrow(X))
  interaction <- X[,2]*X[,3]
  model_matrix <- cbind(intercept, X, interaction)
  # Then compute the A criteria for the candidate design
  return(sum(1/eigen(t(model_matrix)%*%model_matrix)$values))
}

# Following the example of Peter Goos and Brad Jones, this function
# uses a maximization perspective to find the optimal model matrix.
coordinate_exchange <- function(n_factors, n_trials, npasses, testvec, crit){

  # Instantiate a matrix of random values
  n_rows = n_trials
  n_cols = n_factors
  X <- matrix(c(runif(n_rows*n_cols, min=-1, max=1)), nrow=n_rows, ncol=n_cols)
  # Compute the D optimality score until the number of trials has been
  # reached or until there is no more improvement
  trial <- 0
  testvalues <- testvec

  while(trial < npasses){

    candidate_matrices = lapply(1:length(testvalues),
                                matrix, data= NA, nrow=n_rows, ncol=n_cols)
    Xprevious <- X
    # This thingy exchanges the coordinates
    for(i in 1:n_rows){
      for(j in 1:n_cols){
        for(val in 1:length(testvalues)){
```

```r
          xtest <- X
          xtest[i, j] <- testvalues[val]
          candidate_matrices[val][[1]] <- xtest

        }

        if(crit == "D"){
          # This thingy determines which matrix in the list of candidate matrices
          # has a higher D score
          d_scores <- unlist(lapply(candidate_matrices, compute_D))
          max_index <- which.max(d_scores)

          # Now update if the matrix with one element swapped has a greater D score
          if(compute_D(X) < compute_D(candidate_matrices[[max_index]])){
            X <- candidate_matrices[[max_index]]
          }
        }

        if(crit == "A"){
          a_scores <- unlist(lapply(candidate_matrices, compute_A))
          min_index <- which.min(a_scores)

          # Now update if the matrix with one element swapped has a smaller A score
          if(compute_A(X) > compute_A(candidate_matrices[[min_index]])){
            X <- candidate_matrices[[min_index]]
          }
        }
      }
    }
    trial = trial + 1

    # Break out of the loop if no change has been made during the last pass
    if(trial > 1 && identical(X[[1]], Xprevious[[1]])){
      break
    }
  }
  return(X)
}

###############
## PROBLEM 1 ##
###############


c1 <- c(1,1,-1,-1,1,1,-1,-1,1,-1,-1,1)
c2 <- c(1,-1,1,-1,1,-1,-1,1,1,1,-1,-1)
c3 <- c(-1,-1,1,-1,1,1,-1,-1,-1,-1,1,-1)
c4 <- c(1,1,1,1,-1,1,-1,-1,-1,1,-1,-1)
c5 <- c(-1,1,1,-1,-1,1,1,1,1,-1,-1,-1)
c6 <- c(1,-1,-1,1,1,1,1,1,1,-1,-1,-1,-1)

mat <- matrix(c(c1, c2, c3, c4, c5, c6), byrow = FALSE, nrow = 12, ncol = 6)
compute_D(mat)
```

```r
###############
## PROBLEM 2 ##
###############

set.seed(1234)
num_iter = 2000
testvalues = c(-1, 1)
num_factors = 6
num_trials = 12
candidate_designs = lapply(1:num_iter,
                               matrix, data= NA, nrow=num_trials, ncol=num_factors)
dscores <- rep(0, num_iter)
run_simulation <- function(iterations){

  for(i in 1:iterations){
    candidate_designs[[i]] <- coordinate_exchange(num_factors,
                                                  num_trials, 5, testvalues, "D")
    dscores[i] <- compute_D(candidate_designs[[i]])
  }
  return(list(unlist(dscores), candidate_designs))
}

result <- run_simulation(num_iter)
max_index <- which.max(result[[1]])
efficiencies <- unlist(lapply(result[[1]],function(x){
  x/compute_D(mat)*100
  }))
D_optimal_matrix <- result[[2]][max_index][[1]]
boxplot(efficiencies, main = "Candidate Matrix D-Score Relative Efficiencies",
        ylab ="Relative Efficiency", col = "blue")

###############
## PROBLEM 3 ##
###############

set.seed(1234)
num_iter = 2000
testvalues = c(-1, 1)
num_factors = 6
num_trials = 12
candidate_designs = lapply(1:num_iter,
                               matrix, data= NA, nrow=num_trials, ncol=num_factors)
ascores <- rep(0, num_iter)
run_simulation <- function(iterations){

  for(i in 1:iterations){
    candidate_designs[[i]] <- coordinate_exchange(num_factors,
                                                  num_trials, 2, testvalues, "A")
    ascores[i] <- compute_A(candidate_designs[[i]])
  }
  return(list(unlist(ascores), candidate_designs))
}
```

```r
result <- run_simulation(num_iter)
min_index <- which.min(result[[1]])
efficiencies <- unlist(lapply(result[[1]],function(x){
  compute_A(mat)/x*100
  }))

A_optimal_matrix <- result[[2]][min_index][[1]]
boxplot(efficiencies, main = "Candidate Matrix A-Score Relative Efficiencies",
        ylab ="Relative Efficiency", col = "red")

###############
## PROBLEM 4 ##
###############

D_xA <- compute_D(A_optimal_matrix)
A_xD <- compute_A(D_optimal_matrix)

###############
## PROBLEM 5 ##
###############

set.seed(1234)
num_iter = 500
testvalues = seq(-1, 1, length.out = 21)
num_factors = 6
num_trials = 12
candidate_designs = lapply(1:num_iter,
                               matrix, data= NA, nrow=num_trials, ncol=num_factors)
dscores <- rep(0, num_iter)
run_simulation <- function(iterations){

  for(i in 1:iterations){
    candidate_designs[[i]] <- coordinate_exchange(num_factors,
                                                  num_trials, 5, testvalues, "D")
    dscores[i] <- compute_D(candidate_designs[[i]])
  }
  return(list(unlist(dscores), candidate_designs))
}

result <- run_simulation(num_iter)
max_index <- which.max(result[[1]])
efficiencies <- unlist(lapply(result[[1]],function(x){
  x/compute_D(mat)*100
  }))
D_optimal_matrix <- result[[2]][max_index][[1]]
set.seed(1234)
num_iter = 500
testvalues = seq(-1, 1, length.out = 21)
num_factors = 6
num_trials = 12
candidate_designs = lapply(1:num_iter,
                               matrix, data= NA, nrow=num_trials, ncol=num_factors)
ascores <- rep(0, num_iter)
```

```r
run_simulation <- function(iterations){

  for(i in 1:iterations){
    candidate_designs[[i]] <- coordinate_exchange(num_factors,
                                                  num_trials, 2, testvalues, "A")
    ascores[i] <- compute_A(candidate_designs[[i]])
  }
  return(list(unlist(ascores), candidate_designs))
}

result <- run_simulation(num_iter)
min_index <- which.min(result[[1]])
efficiencies <- unlist(lapply(result[[1]],function(x){
  compute_A(mat)/x*100
  }))

A_optimal_matrix <- result[[2]][min_index][[1]]
```